WORKING WITH KUBERNETES YAML MANIFESTS

THERE ARE MULTIPLE LAUNGUAGES IN IT

1. **programming languages**
   (java, python, .net, php)

2. **scripting languages**
   it is a group of commands that we can run to automate the creation and save the time.

3. **markup languages**
   it has set the key value/assign the key value ------> yaml and html

4. yaml ------> yet another markup language
5. html ------> hiper text markup language

6. yml file starts with ----------(---)
7. yml file ends with ----------(...)

EXAMPLE TO WRITE YAML FILES

ADDRESS: - PUNE
- BHAMA CENTRE
- FLOOR NO 5
- CLOUDBLITZ

These is called a multi-line concept that shows these lines are connected to each other.

TWO WAYS TO CREATE A OBJECTS

1. Declarative (using manifest)
2. Imperative (using commands)

SAMPLE POD CREATION USING MANIFEST

1. vim pod.yaml

```
apiVersion: v1
kind: pod
metadata:
  name: nginx
spec:
  container:
  - name: nginxcontainer
    image: nginx
    ports:
    - containerPort: 80
```

Commands:

1. kubectl api-resources ----------> to get the version of the objects
2. kubectl apply -f pod.yaml

SERVICES

Service is used to expose Pods and enable communication inside or outside the Kubernetes cluster.

CREATE A CLUSTER-IP SERVICE USING MANIFEST

1. vim clusterip.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: my-cluster-ip
  labels:
    app: myapp
    env: devops
spec:
  type: ClusterIP
  selector:
    app: myapp
  ports:
```

```
    - name: clusterip-port
      protocol: TCP
      port: 80
      targetPort: 80
```

2. kubectl apply -f clusterip.yaml
3. kubectl get svc

CREATE A NODEPORT SERVICES USING MANIFEST

1. vim nodeport.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: my-nodeport
  labels:
    app: myapp
    env: devops
spec:
  type: NodePort
  selector:
    app: myapp
  ports:
    - name: node-port
      protocol: TCP
      port: 80
      targetPort: 80
```

2. kubectl apply -f nodeport.yaml
3. kubectl get svc

CREATE A LOADBALANCER SERVICE USING MANIFEST

1. vim lb.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: my-lb
```

```
  labels:
    app: myapp
    env: devops
spec:
  type: LoadBalancer
  selector:
    app: myapp
  ports:
    - name: lb-port
      protocol: TCP
      port: 80
      targetPort: 80
```

2. kubectl apply -f lb.yaml
3. kubectl get svc

NOTE:

1. Port means the Service port where the Service exposes the application.
2. TargetPort means the port on which the application inside the pod is running.

LABELS

Labels are key-value pairs attached to Kubernetes objects like Pods, Deployments, Services, Nodes, etc.

1. vim pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
  labels:
    app: nginx
    env: dev
spec:
  containers:
  - name: nginx
    image: nginx
```

2. kubectl apply -f pod.yaml
3. kubectl get pods --show-labels

TO SHOW PODS SPECIFIC TO LABELS

1. kubectl get pods -l app=nginx

SELECTORS

In Kubernetes, Selectors are used to identify and filter Kubernetes objects like Pods based on their labels.

TYPES OF SELECTORS IN KUBERNETES

1. Equality-Based Selectors
2. Set-Based Selectors

1. EQUALITY-BASED SELECTORS

Used to match labels using:

- =
- ==
- !=

Syntax

1. key=value
2. key==value
3. key!=value

Example:

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
  labels:
    app: nginx
    env: dev
```

1. kubectl get pods -l app=nginx
   {Fetch Pods where label app=nginx}
2. kubectl get pods -l env!=production
   {Fetch Pods where environment is NOT production}

Example of equality-based selector

```
selector:
  app: nginx
```

2. SET-BASED SELECTORS

used for advanced filtering using:

- in
- notin
- exists
- !

Syntax:

1. key in (value1,value2)
2. key notin (value1,value2)
3. key
4. !key

1. kubectl get pods -l 'env in (dev,test)'
   {Select Pods whose env is dev OR test}
2. kubectl get pods -l 'env notin (production)'
   {Select all Pods except production}
3. kubectl get pods -l app
   {Select Pods having label app}
4. kubectl get pods -l '!app'
   {Select Pods not having label app}

Example of set-based selector

```
selector:
  matchExpressions:
    - key: env
      operator: In
```

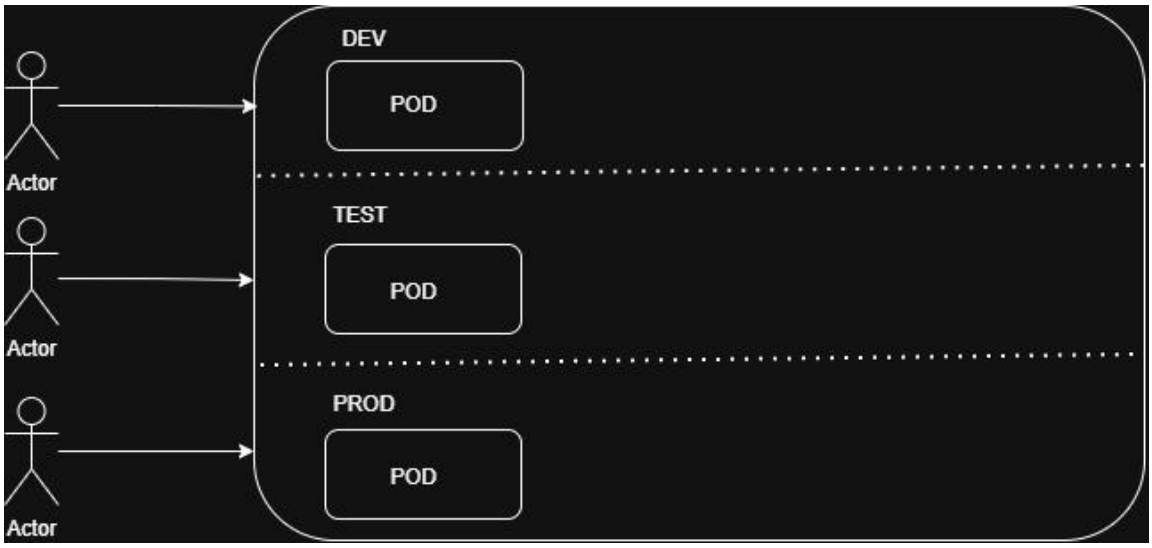
values:
- dev
- test

NAMESPACES

Namespace is a logical partition inside a cluster used to organize and separate resources.

It helps in

- Separate environments like dev, test, prod
- Avoid naming conflicts
- Control access and permissions
- Manage resources easily



COMMAND TO CREATE NAMESPACE

1. kubectl get namespaces
2. kubectl create ns dev
3. kubectl run mypod --image=nginx --port=80 -n dev
4. kubectl get pods -n dev

CREATE NAMESPACE USING MANIFEST

1. vim namespace.yaml
   apiVersion: v1

```
kind: Namespace
metadata:
  name: dev
```

2. kubectl apply -f namespace.yaml
3. kubectl get ns

TO DELETE A NAMESPACE

1. kubectl delete ns dev

REPLICATION-CONTROLLER

1. A ReplicationController (RC) is used to maintain the desired number of pod replicas running in the cluster.
2. If any pod crashes or gets deleted, the ReplicationController automatically creates a new pod.
3. It works on the equality-based selector
4. It is a old-version of creating the replicas of a pod.

5. vim rc.yaml

```
apiVersion: v1
kind: ReplicationController
metadata:
  name: my-rc
  labels:
    app: myapp
    env: devops
spec:
  replicas: 2
  selector:
    app: myapp
  template:
    metadata:
      name: myapp
      labels:
        app: myapp
        env: devops
```

```
  spec:
    containers:
     - name: nginx-container
       image: nginx:latest
       ports:
         - name: myport
           containerPort: 80
```

6. Kubectl apply -f rc.yaml
7. Kubectl get rc
8. Kubectl get pods

HOW TO SCALE REPLICAS

1. kubectl scale rc/my-rc --replicas=5

TO DELETE A REPLICATION CONTROLLER

1. kubectl delete rc my-rc
2. kubectl delete -f rc.yaml

REPLICASET

1. Replicaset is a newest version of creating a replicas of a pod
2. it works on the set-based selectors
3. it simply manages the replicas of a pod running at a given time.

4. vim rs.yaml

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: my-rs
  labels:
    app: myapp
    env: devops
spec:
  replicas: 2
  selector:
    matchLabels:
```

```
    app: myapp
  template:
    metadata:
      name: my-rs-pod
      labels:
        app: myapp
        env: devops
    spec:
      containers:
        - name: nginx-container
          image: nginx:latest
          ports:
            - name: myport
              containerPort: 80
```

5. kubectl apply -f rs.yaml
6. kubectl get pods

TO DELETE A REPLICASET

1. kubectl delete -f rs.yaml
2. kubectl delete rs my-rs
